15,476,027 members Sign in

articles Q&A forums stuff lounge ? Search for articles, questions,

Windows Link (Shortcut) File Explorer



cyprussun
4 Jan 2013 CPOL

Rate me: 4.85/5 (4 votes)

The Link File Explorer analyses Windows shortcut file content and displays the data.

Download source code - 1.15 MB

Download documents - 55.7 KB

Introduction

It was not exactly the flashing action experience I had when I wrote a Side Launcher (Start Menu showing up on the side of the screen) which was seen in a previous Database application written in assembler under Windows. Accessing sequential, unsorted Records, with properly organized record lengths, resulted in speeds near to access indexed databases in any application. Yes, it's true! Reducing the clutter was the way to go and analyzing Link files gave an idea why. ShellLink contributed to the slow acting of the Side Launcher. In fact circumventing Widows, respectively its preference to store huge administrational info into files and deal with that, also seen in PE files, could speed up the application to desired levels. During analysis the Link File Explorer was created.

Features

- The Link File Explorer analyses Windows shortcut file content and displays the data.
- Links can be run from inside the explorer.
- Internet Shortcuts (plain text files) are supported in this application and can be run.
- Link files can be taken into a HexEditor from the Link Explorer.
- Network, Printer, Mouse, etc. link files are supported.
- Accessing Desktop, StartMenu and user directory from inside the Explorer.

- The Link File Explorer contains an easy report facility.
- Report files can be edited from inside the Link File Explorer.
- Opening documents document files directly.
- Related data is supported with color coding for easy recognition.
- A whole variety of documents, explaining structures, are included in the download.
- The Explorer uses default applications for external calls.

Errors

The only error which can occur could be, when you want to start HexEdit, that the file is in use by another application. In that case, exclude the folder “c:\users\...\My Documents” from the virus checker. This is the location where a copy of your link file is going to be saved. HexEdit will not be provided with the original link file from the Link Explorer.

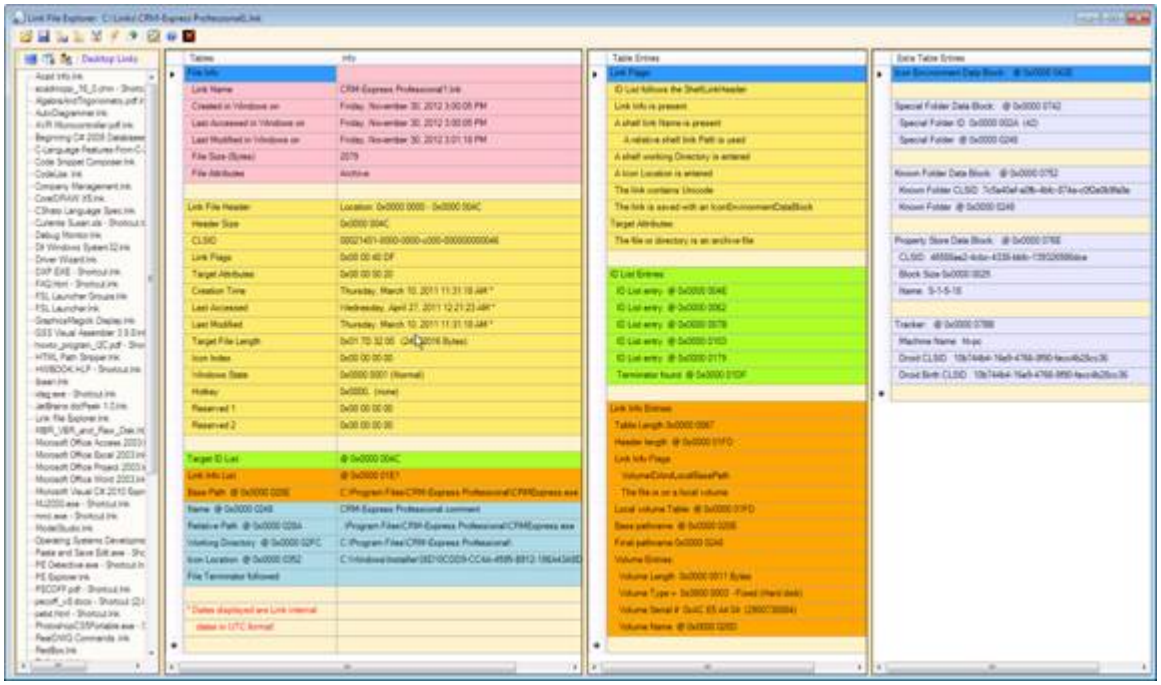
Assumptions

Since an executable is included in the download, minimal knowledge is required to make the application run. For extensions and modifications advanced knowledge of C# and Visual Studio (Express) is required. The developer will operate at the Bit level. There is hardly help and help on help available in the Link Explorer. More information is provided in the additional PDF files located in the doc folder. Microsoft has a wealth of free technical documents for download available now days. Use it for your projects.

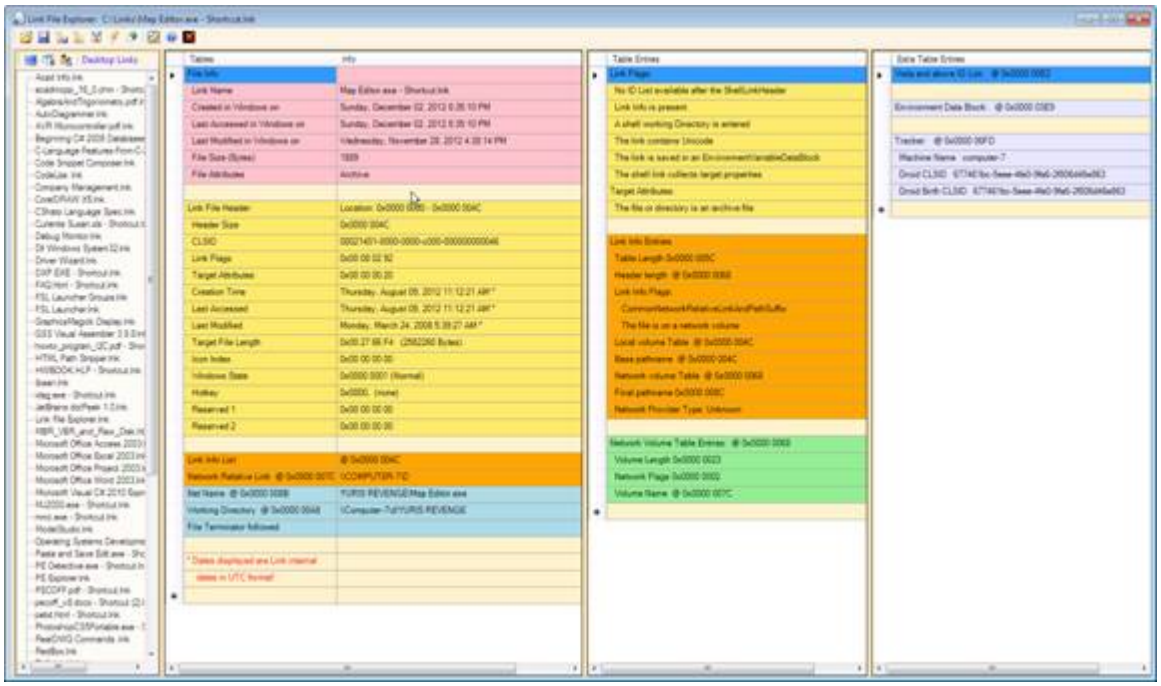
Run or Compile

The download contains executables, all source files besides documentation. Partially documented link files (Word files) are included in the download. These documents where used during analysis of the link files. Be prepared to read some unusual dense written source code in case you intend to alter the functionality.

A local link file

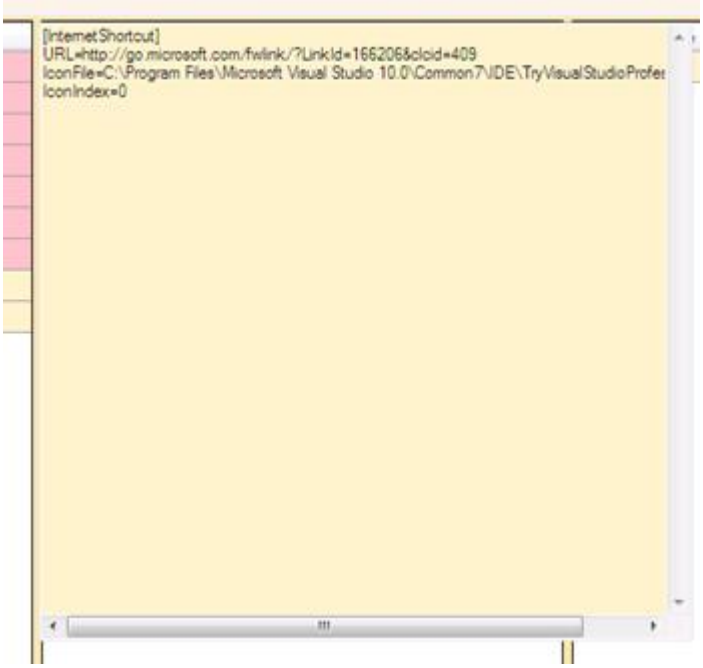


A network link file



Entries in the Extra Table which are displayed in Red and preceded by an asterisk are functions which are not jet completely written. This occurs then a (**Console FE Data Block**) is present in the link file.

A simple window opens for an Internet shortcut which looks like this:



The Code

May o may, do we need this value before doomsday? Are all these values valid? Link files contain many values of a whole variety as: UInt32, Unicode, CLSID, C-String, Date, Flags, arrays, fixed length records, variable length records and of course garbage. Several different terminators are used. The structures and flags can be read up in *[MS-SHLLINK].pdf*. A short extract is shown here.

The basic structure is:

```
Link header             Fixed record length
Target link Info        Variable records having fixed and variable record length
IDList Info             Variable records having variable record length
Link Info               Variable records having fixed and variable record length
String Data             Variable records having variable record length
Extra Data              Variable records having fixed and variable record length
```

Any of the info, except the Link header, is actually optional respectively is not mandatory. This is on a case to case basis and flags in the Link Header are set for the presence or absence of a record. For any structure and flags see *[MS-SHLLINK].pdf* or find some short info in *Structures.cs*.

The main structures are:

The link header structure is:

C#

```
UInt32 headerSize
Guid   CLSID
```

```
UInt32 shortcutFlags
UInt32 targetFlags
FILETIME  creationTime, lastAccess, lastModified
UInt32 fileLength
UInt32 iconIndex
UInt32 windowsState
UInt32 hotkey
UInt32 reserved1, reserved2
```

The target link info structure is:

C#

```
UInt32 LinkInfoSize
UInt32 LinkInfoHeaderSize
UInt32 LinkInfoFlags
UInt32 offsetVolumeTable
UInt32 offsetBasePath
UInt32 offsetNetwork
UInt32 offsetFinalPath

If LinkInfoHeaderSize > 0x1C
UInt32 LocalBasePathOffsetUnicode        (optional)
UInt32 CommonPathSuffixOffsetUnicode     (optional)
```

The `ItemIDList` structure is:

C#

```
UInt32 ListSize
UInt16 ItemIDSize
     Data
UInt16 ItemIDSize
     Data
…..
UInt16 Terminator
```

The local volume table structure is:

C#

```
UInt32 volumeTableLength
UInt32 volumeType
ULONG volumeSerialNbr
UInt32 offsetVolumeName
ULONG  VolumeLabelOffsetUnicode          (optional)
```

The network info structure is:

C#

```
UInt32 netwVolumeLength
UInt32 netwFlags
UInt32 NetNameOffset
```

```
UInt32 DeviceNameOffset
UInt32 NetworkProviderType
UInt32 NetNameOffsetUnicode             (optional)
UInt32 DeviceNameOffsetUnicode          (optional)
```

The actual task is to arbitrate the file content preferably with a `BinaryReader` and marshaling the data buffer before displaying the values.

C# Shrink ▲

```
using System.IO;
using System.Runtime.InteropServices;

FileStream fs;
BinaryReader reader;
GCHandle handle;
Byte[] headerBuffer;
OpenFileDialog openFileDialog = new OpenFileDialog();

// Define the structure of the header
struct lnk_FILE_HEADER                  // You can use a class instead
{   // Length = #4C Bytes
    public UInt32  headerSize;          // Always 4C 00 00 00        The header size
    public Guid     CLSID;              // The current GUID for shortcuts. It may change
in the future.
    public UInt32  shortcutFlags;       // See [MS-SHLLlink].pdf
    public UInt32  targetFlags;         // Target file flags  Flags are explained below
    public FILETIME creationTime;       // (UTC) 100-nanosecond intervals, elapsed since
12:00
                                        //     midnight, January 1, 1601 A.D.
    public FILETIME lastAccess;         // Last access time (UTC)
    public FILETIME lastModified;       // Modification time (UTC)
    public UInt32  fileLength;          // The length of the target file. 0 if the target
is not a file.
    public UInt32  iconIndex;           // If the file has a custom icon (set by the
flags bit 6).
    public UInt32  windowsState;        // 1:Normal Window 2:Minimized 3:Maximized
    public UInt32  hotkey;              // The hot key assigned for this shortcut
    public UInt32  reserved1;           // Always 0
    public UInt32  reserved2;           // Always 0
};

lnk_FILE_HEADER LinkFileHeader = new lnk_FILE_HEADER();

public Form1()
{
    InitializeComponent();
    if(ofd.ShowDialog() == DialogResult.OK)
        readHeader();
}

public  void readHeader()
{
    // Read the header of an opened Link File
    fs = new FileStream(openFileDialog .FileName, FileMode.Open, FileAccess.Read,
FileShare.Read);
```

```
    reader = new BinaryReader(fs);

    headerBuffer = reader.ReadBytes(Marshal.SizeOf(typeof(lnk_FILE_HEADER)));
    handle = GCHandle.Alloc(headerBuffer, GCHandleType.Pinned);
    LinkFileHeader = (lnk_FILE_HEADER)Marshal.PtrToStructure(handle.AddrOfPinnedObject(),
                                typeof(lnk_FILE_HEADER));

    // Display the data values now.
    ……..
    ……..
    ……..

    fs.Close();
    fs.Dispose();
    reader.Close();
    reader.Dispose();
}
```

The source contains unusual packed code as mentioned before. Packed code just gives a better overview over the whole function versus spread code which could be better readable for details. Let's put it all down to taste. Anyway, the code is well and clear documented and does not contain any unmanaged code.

The project is split up in following code parts:



```
Form1.cs                The main part
UserInterface.cs        Whatever has to do with user interaction
Subroutines.cs          All general functions
Conversions.cs          Bin, Hex… conversions, good for other projects as well
ExtraTables….cs         Deals with all extra tables in the link, found after strings
Structures.cs           The definition of all structures and some records
About.cs
```

A dependency graph, where all the related functionality shows, is included in the source. It's a bit too large to be displayed here. The diagram can be viewed as a graphic in any Visual Studio like 2011, 2011 and 2012. Visual Studio Express shows it as a DGML text file.

You might want to use some of the source for your own projects. The source is split up into logical segments what supports the reuse of code bits or even files. This also makes it easy to transfer code bits into another project from the author called the Code Snippet Composer. More about the C# composer another time.

Remark: Dates in the link header conflict with dates obtained through Windows. Strings can occur as Unicode and or terminated (c_str()) strings. Structures often end with UInt32 0. The code was written with Visual Studio Express and tested under Windows 7. There should not be any trouble to run the Explorer under any other Windows platform. Worst case scenario, recompile it. No third party components where used but HexEdit from http://www.hexedit.com or http://www.hexeditpro.com (see also http://www.codeproject.com "HexEdit - Window Binary File Editor" by Andrew Phillips) was

included for additional support. HexEdit is located in a subdirectory of the Link File Explorer. The source might still be obtainable.

A pane for directly accessing Links in folders like Desktop, Startmenu and a user selected folder shows on the left. Double clicking the link executes the analysis of the link file content.

No Menu?

No, there are ToolStrip buttons for direct accessing the provided functions. Hints show their actions. That's faster, easier, obvious and way more comfortable.

Shortcuts make actions easy to access:



```
F1                  Help or About
F2 or <Ctrl> + O    Open a link file
<Ctrl> + S          Save a report text file
<Ctrl> + E          Edit last saved report
<Ctrl> + R          Edit a report read form the disk
<Ctrl> + H          Show the link file in HexEdit
<Ctrl> + X          Execute or run the link
<Ctrl> + I          Start the Internet Explorer with the link
<Ctrl> + Q          Quit the Application
```

Double Features

During test different dates have shown up. Internal link file dates do not match date reports from Windows. Anyway, both are displayed and it's up to the user preference which shall be accepted as valid. Is that confusing forensics? Guessing, Link internal dates are not updated.

Improvements

Yes, there is probably still some bad code in the source and not all extra information is completely supported yet (due to lack of data). Feel free to modify it. Over 250 link files did run against the Explorer without showing any troubles (sort of a job for a monkey, next time, affirmative). Areas of interest are: `CONSOLE()`, `DARWIN()`, `ENVIRONMENT()`, `ICON_ENVIRONMENT()`, `KNOWN_FOLDER()`, `SPECIAL_FOLDER()`, and `VISTA_AND_ABOVE_IDLIST()`. All this areas can be found in *ExtraTablesSubroutines.cs* and contain mainly duplicate data which is already displayed or data which does not make much sense to be displayed.

Further releases

There is no intention to alter the source in any way since the results are already beyond expectation. Feel free to mail updates so they can be posted on site.

Contributors

- Andrew Phillips Hexedit, CodeProject HexEdit - Window Binary File Editor

Prerequisites

- .NET Framework (4.5)
- Hexedit installed if this option is going to be used.

Warranty

There is absolute no warranty what so ever implied or assumed. Use it at your own risk. It does a marvelous job for the author. Copyrights and Trademarks shall belong to their respective owners. I am not going to fight over that!

Epilog

Have you ever thought that a simple click on Create Shortcut in the explorer creates such a complex link? Are you bored or do you want to learn something? Analyze Links or PE files! Good luck.

Further reading

- http://msdn2.microsoft.com/en-us/library/E4BD6494-06AD-4aed-9823-445E921C9624
- [MS-SHLLINK].pdf: Shell Link (.LNK) Binary File Format”
- [MS-PROPSTORE].pdf: "Property Store Binary File Format"
- [MS-DFSNM].pdf: "Distributed File System (DFS): Namespace Management Protocol”
- [MS-DTYP].pdf: "Windows Data Types"
- [MS-LCID].pdf: "Windows Language Code Identifier (LCID) Reference"
- [RFC2119].pdf : "Key words for use in RFCs to Indicate Requirement Levels"
- [RFC5234].pdf: "Augmented BNF for Syntax Specifications: ABNF",
- [MS-DLTW].pdf: “Distributed Link Tracking Workstation Protocol”
- [MS-GLOS].pdf: “Windows Protocols Master Glossary”
- The Windows Shortcut File Format by Jesse Hager

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

cyprussun

Philippines

Grew up in a metal processing company and did industrial HW/SW development since the birth of Intel's 8080. Lectured IT since 1986 at several levels. Hobbies, sidesteps: Woodworking and deep sea diving. Background: ASM, C, C++. Platforms: Win, Novel, CP/M, MP/M, DOS, (Linux).

It's not the developer's duty to pay up for the engineer's ignorance.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

Wrong internal dates
b_jones2@optusnet.com.au 20-Mar-14 21:56

Complete Project Download
cyprussun 12-Mar-13 19:09

Thanks jfriedman for your comment!
cyprussun 9-Feb-13 20:44

My vote of 3
jfriedman 4-Jan-13 18:16

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2013 by cyprussun
Everything else Copyright © CodeProject, 1999-2022

Web02 2.8:2022-10-01:1